CAMPUSTRACK – CENTRALIZED LOST AND FOUND PLATFORM FOR COLLEGE CAMPUSES

A PROJECT REPORT

Submitted By :

Suryakant Jaiswar

Seat No : 510512446

Bachelor in Computer Science (Third Year)

Under the Guidance of

RAGVINDER KAUR

(Assistant Professor)

Achievers College of Commerce and Management,

Kalyan (W)

(Affiliated to University of Mumbai)

ACADEMIC YEAR: 2026 – 2027

DECLARATION

We hereby declare that the project entitled **"CampusTrack: A Secured Centralized Lost and Found Platform for College Campuses"** submitted by us in partial fulfilment of the requirements for the **Third Year Bachelor of Science in Computer Science (T.Y.B.Sc. C.S.)** is our original work carried out under the guidance of **Ms. Ragvinder Kaur Chadha** at **Achievers College of Commerce and Management, Kalyan (W)**.

We further declare that this project has not been submitted, either in whole or in part, to any other university or institution for the award of any degree or diploma. The information, technical documentation, architectural models, and references used in this project have been duly acknowledged.

We take full responsibility for the authenticity and originality of the software design, source code implementation, and testing methodologies presented in this project report.

Project Title: CampusTrack: A Secured Centralized Lost and Found Platform for College Campuses

Name of Student : Suryakant Jaiswar

Seat Number: 510512446

Class : T.Y.B.Sc. Computer Science

Academic Year: 2026–2027

Place: Kalyan

Date: 30/10/2026

Signature of Student :

_____

(Suryakant Jaiswar)

Project Guide :

Ms. Ragvinder Kaur Chadha

Signature of Project Guide :

_____

ABSTRACT

CampusTrack is an enterprise-grade, centralized, and secured lost and found web platform engineered specifically for collegiate institutions to eliminate the chronic fragmentation, lack of verification, and permanent personal property loss prevalent across expansive university grounds. Built using a modern full-stack decoupled architecture comprising **React 19**, **Tailwind CSS v4**, **Node.js Express 5**, and an embedded **SQLite** database managed through **Prisma ORM**, CampusTrack replaces insecure, manual noticeboards and unmoderated social media messaging threads with a verified digital recovery workstation.

The platform introduces a novel *Anti-Fraud Security Challenge Protocol*, wherein individuals reporting found articles can establish concealed challenge questions (e.g., specific internal engravings, serial number fragments, or unique lock-screen backgrounds) that prospective claimants must accurately answer before physical handover can be authorized. Coupled with server-side multipart photographic ingestion using Multer, JSON Web Token (JWT) stateless authorization, university roll number validation, in-app notification dispatchers, and a comprehensive administrative campus security oversight console, CampusTrack establishes a dependable, verifiable, and auditable asset recovery lifecycle.

> **Final Project State & Release Readiness**
>
> The production release (v1.0.0) has been successfully consolidated, tested, and published to GitHub. The live application was validated end-to-end across multiple device viewports, verifying sub-second multi-category search responses, secure image streaming, dispute resolution mechanisms, and authenticated student session workflows with zero unresolved critical runtime vulnerabilities.

Keywords:

Lost and Found Management System, React 19, Node.js Express 5, Prisma ORM, SQLite Relational Persistence, Anti-Fraud Verification, Campus Security Automation, Single Page Application (SPA), Full-Stack Engineering.

TABLE OF CONTENTS

Chapter	Topic	Page No.
-	**Official Declaration**	ii
-	**Abstract**	iii
1	**Introduction**	1
2	**Problem Definition**	6
3	**Objectives**	9
4	**Requirement Specification**	12
5	**Existing System**	17
6	**Proposed System**	20
7	**System Analysis & Design**	27
8	**Methodology**	39
9	**System Implementation**	44
10	**Testing & Verification**	52
11	**Results / Screenshots**	54
12	**Conclusion**	59
13	**Future Scope**	61
14	**References**	63
App.	**Technical Appendices**	64

CHAPTER-1

1. INTRODUCTION

1.1 Introduction

1) Background

Higher education institutions, colleges, and university campuses represent dynamic micro-ecosystems characterized by high population density, constant physical transit, and diverse infrastructure. On any given academic day, thousands of undergraduate students, postgraduate researchers, faculty members, lab assistants, and administrative staff move between multi-story academic wings, computerized laboratories, central libraries, cafeterias, auditoriums, and sports facilities.

During these routine transitions, personal belongings are inevitably forgotten, misplaced, or inadvertently exchanged. Misplaced items span a wide spectrum of utility and financial worth:

- **Critical Academic Credentials:** Official university identity cards, library barcoded passes, hall tickets, and syllabus laboratory journals.
- **High-Value Electronic Assets:** Scientific programmable calculators, laptop power chargers, wireless Bluetooth earbuds, portable SSDs, and pen drives containing semester source code.
- **Personal Valuables:** Wallets, government identity cards (Aadhaar cards, PAN cards, driving licenses), spectacles, motorcycle keys, and branded apparel.

When an asset such as a scientific calculator or university hall ticket is lost hours before a practical or terminal semester examination, the resulting anxiety and operational disruption are severe. The student is forced to abandon exam preparation to frantically tour campus buildings, interrogate security guards, and post fragmented pleas across informal messaging groups.

2) Domain Challenges

Historically, tertiary educational institutions manage misplaced belongings through decentralized, ad-hoc, and manual mechanisms:

- **Paper Registers at Gatehouses:** Security guards maintain handwritten registers where finders log turned-in items. These logbooks lack photographic records, provide zero search capabilities, and frequently suffer from illegible handwriting.
- **Notice Boards:** Physical paper notices pinned to corkboards in main corridors tear, get covered by club flyers, and fail to provide immediate notifications to owners.
- **Informal Social Media Groups:** Students frequently post about lost articles in unofficial WhatsApp or Telegram batch chats. However, these notices quickly get buried beneath hundreds of casual

messages, expose personal mobile numbers to harassment, and provide zero verification mechanisms to stop dishonest individuals from claiming items that do not belong to them.

3) Institutional Context

To resolve these institutional bottlenecks, **CampusTrack** was conceived and engineered. CampusTrack is an automated, web-based centralized lost and found workstation designed to provide a transparent, accountable, and fraud-resistant recovery workflow for Achievers College of Commerce and Management.

1.2 What is the Project About?

CampusTrack is a modern web application engineered to coordinate the complete reporting, indexing, verification, and recovery pipeline for lost and found property across campus grounds.

Rather than acting as a passive bulletin board, CampusTrack serves as an active verification mediator. It introduces an **Anti-Fraud Security Challenge Protocol**. When a student discovers an abandoned item (such as a lost wallet, mobile phone, or keys), they post the record along with a concealed challenge question—such as *"What is written on the keychain tag?"* or *"What is the brand and color of the inner pouch?"*. Prospective claimants cannot view contact information or take possession of the item until they submit a matching answer and supporting photographic proof that the finder or campus security officer validates.

1.3 Why Was This Topic Selected?

The selection of "CampusTrack" as our final-year engineering project is rooted in two fundamental pillars: solving an immediate tangible real-world problem and applying cutting-edge full-stack software engineering principles.

1) To Solve a Tangible Real-World Problem

Misplacing valuable personal property is an experience shared by virtually every university student. The absence of an organized recovery system leads to:

- Permanent financial loss for students from modest economic backgrounds.
- Privacy violations stemming from public broadcasts of personal contact numbers.
- Opportunistic theft and fraudulent claims in unmoderated group chats.
- Accumulation of unclaimed, uncataloged property in campus security lockers.

2) To Build a Production-Grade Web Application

From an academic and technical standpoint, the project provided an ideal vehicle to implement enterprise-level software design patterns:

- **Component-Driven Single Page Architecture:** Leveraging React 19 and Tailwind CSS v4 to deliver an instantaneous, responsive user interface.
- **Stateless RESTful API Backend:** Utilizing Express 5 and Node.js with JSON Web Token (JWT) authorization guards and Bcrypt password hashing.
- **Embedded Relational Data Modeling:** Employing Prisma ORM with SQLite, enabling zero-configuration, self-contained relational database portability.
- **Multipart Binary Media Pipelines:** Streaming and validating photographic evidence directly to disk storage using Multer middleware with MIME type guards.

1.4 Importance of CampusTrack

CampusTrack converts an uncertain, informal activity into a standardized, auditable recovery pipeline:

```
Incident Occurs (Lost/Found)
          |
          ▼
Structured Listing Submission (With Photos & Anti-Fraud Question)
          |
          ▼
Instant Full-Text & Category Public Indexing
          |
          ▼
Claimant Files Verification Answer & Document Proof
          |
          ▼
Poster Evaluates Claim (Approve / Reject Action)
          |
          ▼
Verified In-Person Handover at Campus Security Desk
```

1.5 Brief Overview of the Proposed Solution

CampusTrack provides a multi-role web ecosystem comprising student portals and a security desk console. Key capabilities include:

- **Student Authentication & Identification:** University roll number and college email validation ensuring institutional accountability.
- **Dual Reporting Pipelines:** Dedicated flows for reporting lost articles versus registering found property.
- **Multi-Dimensional Search & Filtering:** Real-time keyword matching across titles, descriptions, categories, and campus locations (e.g., Computer Lab 3, Central Library, Canteen).

- **Claim Resolution Engine:** Structured review modals enabling posters to evaluate ownership proof without exposing private contact info.
- **Campus Administration & Security Console:** Comprehensive dashboard displaying platform metrics, audit logs, and listing moderation tools.

1.6 Project at a Glance

Attribute	Project Specification Details
Project Title	CampusTrack – Centralized Lost and Found Platform for College Campuses
Release Version	v1.0.0 (Production Hardened & Fully Operational)
Author / Student	Suryakant Jaiswar (Seat No: 510512446)
Degree / Class	Bachelor in Computer Science (Third Year) / T.Y.B.Sc. C.S.
Project Guide	Ms. Ragvinder Kaur Chadha (Assistant Professor)
Institution	Achievers College of Commerce and Management, Kalyan (W)
Affiliation	Affiliated to University of Mumbai
Academic Year	2026 – 2027
Repository	github.com/proot-distrologinubuntu275/Findit
Frontend Technologies	React 19, Vite 8, Tailwind CSS v4, Lucide React, Axios
Backend Technologies	Node.js (v18+), Express.js (v5.3.0), Multer (v2.5.0), JWT (v9.0.3), BcryptJS
Database & ORM	SQLite 3 (dev.db) managed with Prisma ORM (v6.4.1)
Target Operating System	Cross-Platform (Windows, Linux, macOS, iOS, Android Browser)

CHAPTER-2

2. PROBLEM DEFINITION

2.1 Problem Definition & Context

In collegiate environments, the recovery of misplaced personal property is hampered by a lack of central coordination, severe information fragmentation, and the absence of verification protocols. When an item is lost, the victim has no single authoritative location to query. Conversely, when an honest individual finds an article, they have no secure, privacy-preserving method to locate the legitimate owner.

1) Information Asymmetry & Fragmentation

In typical campus operations, reports of found belongings are scattered across disparate gatehouses, faculty rooms, and departmental offices. If a student leaves a calculator in the 3rd Floor Electronics Lab, a peon may deposit it with the department clerk, while the student searches the ground-floor security gate. Because these distinct physical points maintain isolated paper notebooks, the item remains unclaimed indefinitely.

2) False Claims and Opportunistic Fraud

When students resort to broadcasting lost or found announcements on public messaging channels (such as college WhatsApp batches or Instagram confession pages), unscrupulous individuals exploit the situation. For instance, when a found wireless earbud case is broadcast publicly, any member can privately message the finder claiming ownership. Because finders lack formal investigative guidelines, items are routinely handed over to the first claimant rather than the rightful owner.

2.2 Who Faces the Problem?

- **Undergraduate & Postgraduate Students:** Face severe financial and academic consequences when calculators, identity cards, or project laptops are lost.
- **Teaching Faculty & Lab Staff:** Regularly lose proprietary storage drives, lecture presentation notes, and specialized lab equipment during classroom rotations.
- **Campus Security Staff:** Suffer operational bottlenecks trying to catalog physical property lockers with inadequate, illegible paper ledgers.
- **College Administration:** Faces reputational damage and student grievances regarding perceived lack of campus safety and security.

2.3 Limitations of Current Manual Approaches

Approach	Operational Mechanism	Core Deficiencies & Failure Modes
Physical Security Registers	Handwritten logs at college gatehouses.	No keyword searchability; entries cannot be cross-referenced across gates; zero photo capture; logs degrade over time.
Corridor Notice Boards	Paper posters pinned to corridor corkboards.	Low visibility; flyers get torn or covered; static information cannot be updated once recovered; zero privacy protection.
Social Media Batches	Unmoderated WhatsApp, Telegram, or Instagram posts.	Notices vanish beneath chat spam; exposes personal phone numbers to unsolicited calls; vulnerable to opportunistic theft.

2.4 Why Is the Project Needed?

Campus communities urgently require an authoritative, structured, and institutional platform that guarantees:

- **Instantaneous Campus-Wide Indexing:** Allowing any student to search across all departments and locations within milliseconds.
- **Verifiable Ownership Claims:** Guaranteeing that claimants must prove ownership before receiving items.
- **Student Data Privacy:** Ensuring personal contact info is never publicly exposed to unauthorized users.
- **Administrative Auditing:** Giving college authorities visibility into lost item recovery metrics and dispute resolutions.

2.5 How the Project Helps

CampusTrack provides a structured digital workspace that replaces ad-hoc guessing with automated verification. By enforcing structured schemas (category, location, date, time, photographic proof, and anti-fraud challenge questions), the system eliminates ambiguity and establishes trust between finders and owners.

CHAPTER-3

3. OBJECTIVES

3.1 Project Objectives

The primary software engineering and operational objectives of CampusTrack are:

1. **Centralized Academic Hub:** Build a responsive web application serving as the single point of truth for lost and found items across the entire college campus.
2. **Dual Reporting Pipelines:** Develop independent, intuitive workflows for submitting both "Lost Item" notifications and "Found Item" registrations.
3. **Anti-Fraud Verification Protocol:** Design an interactive challenge-question verification engine preventing dishonest claims of valuable electronics and personal effects.
4. **Server-Side Multipart Media Handling:** Implement a robust image processing pipeline with Multer to store and serve high-resolution item photographs on disk.
5. **Stateless Security & Authorization:** Implement JSON Web Token (JWT) authentication and Bcrypt password hashing to protect private user data and API routes.
6. **Sub-Second Search & Multi-Filter Querying:** Engineer high-speed client and server filtering across categories, campus buildings, and item status.
7. **Interactive Claim Management:** Allow item posters to review answers, inspect uploaded purchase bills or proof images, and approve or reject claims.
8. **Campus Security Administration Console:** Deliver a specialized administrative dashboard enabling security personnel to oversee all listings, resolve disputes, and delete spam.
9. **Zero-Configuration Relational Persistence:** Utilize SQLite 3 with Prisma ORM to ensure complete project portability across local development and hosting servers.
10. **Production Governance & Versioning:** Deliver a release (v1.0.0) hosted on GitHub with comprehensive documentation and automated runners.

3.2 Scope of the Project (In Scope)

Functional Subsystem	Included Engineering Features & Capabilities
User Management	Registration with student roll number, college email validation, Bcrypt password encryption, JWT session management, profile dashboard.
Item Catalog & Ingestion	Dual report forms, campus location tagging (e.g., Library, Lab 2, Canteen), category taxonomy (Electronics, IDs, Wallets, Books), image uploads.
Anti-Fraud Engine	Concealed challenge questions, answer submission modal, optional proof image attachments, poster approve/reject decision workflow.
Search & Filtering	Live real-time client-side search, debounced text search, category pills, status tabs (Lost, Found, Claimed, Resolved).
Security Console	Admin authorization, system metrics summary, item listing moderation, user account directory, and audit logs.

3.3 Outside the Current Release

To maintain strict quality boundaries within the academic lifecycle, the following items are intentionally excluded from v1.0.0:

- Facial recognition or biometric matching of students.
- Automated drone delivery of returned personal property.
- Inter-university federated synchronization across multiple external campuses.

3.4 Completion Criteria

The project was deemed complete upon achieving:

- Zero open unhandled runtime exceptions across all REST API endpoints.
- 100% successful pass rate on all automated functional smoke tests and claim verification cycles.
- Complete deployment readiness with automated seed data and cross-platform Node.js startup scripts.

CHAPTER-4

4. REQUIREMENT SPECIFICATION

4.1 Hardware Requirements

Hardware Component	Minimum Operational Specification	Recommended Production Specification
Client Processor	Dual-Core 1.6 GHz x86/ARM Processor	Quad-Core 2.4 GHz Intel Core i5 / Apple M-Series
Client Memory (RAM)	2 GB System Memory	4 GB to 8 GB System Memory
Client Display	360x640 Mobile Touch Display	1920x1080 Full HD Display
Server Hardware	1 vCPU, 1 GB RAM, 10 GB Disk (VPS or Local PC)	2 vCPU, 4 GB RAM, 40 GB SSD Storage
Network Connection	Standard Campus Wi-Fi / 4G Cellular (1 Mbps)	High-Speed Campus LAN / 5G Connection (10+ Mbps)

4.2 Software Requirements

Layer	Software Component	Version & Architectural Role
Runtime	Node.js	v18.0.0+ LTS (JavaScript Server Execution Engine)
Backend Framework	Express.js	v5.3.0 (HTTP RESTful Microservices & Middleware)
Database Engine	SQLite 3	Embedded Relational ACID-Compliant Database File
ORM	Prisma ORM	v6.4.1 (Schema Modeling, Migrations, Client Engine)
Media Handler	Multer	v2.5.0 (Multipart/Form-Data Image File Streaming)
Security Tokens	JSON Web Token (jsonwebtoken)	v9.0.3 (HMAC SHA-256 Stateless Bearer Tokens)
Cryptography	BcryptJS	v3.0.3 (Adaptive Salting & Password Hashing)
Frontend Library	React	v19.0.0 (Declarative UI Component Architecture)
Build Bundler	Vite	v8.0.0 (Fast HMR & Optimized Production Build Tool)
Styling Framework	Tailwind CSS	v4.0.0 (Utility-First Responsive Design Engine)
Icons	Lucide React	v0.400+ (Lightweight Vector SVG Icon Library)
Version Control	Git & GitHub	Distributed Versioning & GitHub Cloud Repository

4.3 Functional Requirements

Req. ID	Feature Scope	Requirement Description & System Behavior
FR-01	Student Registration	The system shall register new users with name, valid email, student ID / roll number, and password, ensuring unique email constraints.
FR-02	Password Security	The system shall hash all passwords using Bcrypt with 10 salt rounds prior to persisting them to the database.
FR-03	User Authentication	The system shall authenticate credentials and issue a signed, tamper-proof JWT bearer token expiring in 24 hours.
FR-04	Lost Item Reporting	The system shall allow authenticated students to file a lost item application with title, category, location, date, and description.
FR-05	Found Item Registration	The system shall permit users to report found articles, upload photos via Multer, and set an anti-fraud challenge question.
FR-06	Live Catalog Search	The system shall provide instantaneous full-text filtering across all active listings by keyword, campus building, and item type.
FR-07	Anti-Fraud Claim Filing	The system shall allow users to file a claim by answering the challenge question and attaching optional proof documents.
FR-08	Claim Evaluation	The item poster shall have exclusive access to view submitted claims and execute "Approve" or "Reject" status transitions.
FR-09	Notification System	The system shall generate automated in-app notifications to claimants when their claim is approved, rejected, or updated.
FR-10	Resolution Tracking	When a claim is approved, the item status shall transition to "CLAIMED" or "RESOLVED", removing it from active searches.
FR-11	Administrative Oversight	The system shall provide an admin console allowing security personnel to delete spam, view analytics, and inspect audit logs.
FR-12	Profile Management	The system shall allow users to view their filed listings, active claims, and manage account details in a personal dashboard.

4.4 Non-Functional Requirements

- **Performance:** Public catalog queries and full-text searches shall return results in less than 300 milliseconds under local server execution.
- **Security:** All REST API endpoints modifying state shall enforce JWT token validation and role-based permissions (Student vs Admin).

- **Portability:** The entire application, backend API, and database shall execute seamlessly on Windows, Linux, and macOS without requiring external cloud database subscriptions.
- **Responsiveness:** The UI layout shall dynamically adapt to screen resolutions ranging from 320px mobile smartphones up to 4K ultra-wide monitors.
- **Maintainability:** Clean decoupling of frontend client code, backend REST controllers, and Prisma schema definitions ensures effortless code maintainability.

CHAPTER-5

5. EXISTING SYSTEM

5.1 Existing System Overview

In prevailing educational environments, the handling of lost and found property is entirely informal and manual. Institutions depend on security guard handbooks, corkboards, and unstructured social media threads.

5.2 Problems in the Existing System

- **No Centralized Storage or Indexing:** An item found in the Science Laboratory remains unknown to security personnel at the Main Entrance Gate.
- **High Susceptibility to Opportunistic Theft:** In open WhatsApp groups, opportunistic individuals falsely claim lost valuables because finders cannot verify true ownership.
- **Privacy Compromise:** Students post their personal mobile phone numbers on physical flyers, leading to unsolicited calls and privacy violations.
- **Lack of Visual Documentation:** Handwritten paper logbooks cannot store photographs, resulting in ambiguous entries like "black bag" that confuse students.

5.3 Comparative Evaluation Matrix

Evaluation Metric	Manual Security Book	College Chat Groups	CampusTrack (Our System)
Searchability	❌ None (Manual leafing)	⚠️ Poor keyword scrolling	✅ **Instant Real-Time Search**
Photo Storage	❌ Impossible	⚠️ Unindexed image spam	✅ **High-Res Linked Photos**
Ownership Proof	❌ None	❌ None (First to claim wins)	✅ **Anti-Fraud Challenge Question**
Student Privacy	⚠️ Public ledger view	❌ Mobile numbers exposed	✅ **Mediated Secure Contact**
Campus Coverage	❌ Single physical gate	⚠️ Fragmented batch chats	✅ **Unified Campus-Wide Portal**
Administrative Audit	❌ Incomplete records	❌ Zero institutional audit	✅ **Complete Relational Audit Log**

CHAPTER-6

6. PROPOSED SYSTEM

6.1 Proposed System Overview

CampusTrack establishes an end-to-end digital recovery infrastructure. It unifies students, faculty, and campus security officers into a single, cohesive ecosystem.

6.2 Key Architectural Decisions

- **Modern Single Page Application:** Built using React 19 and Tailwind CSS v4, providing an app-like experience with immediate UI feedback and zero page reloads.
- **Decoupled RESTful API:** An Express 5 backend delivering clean JSON endpoints, allowing future native mobile apps to connect to the exact same backend.
- **Embedded Relational Database:** SQLite managed via Prisma ORM provides ACID-compliant transactions with zero external database configuration.
- **Local Multipart Image Streaming:** High-resolution photos are handled via Multer and served locally, eliminating expensive third-party storage fees.

6.3 Anti-Fraud Challenge Workflow

The distinguishing engineering feature of CampusTrack is its Anti-Fraud Security Challenge Protocol:

```
1. Finder Discovers Item (e.g. Branded Wallet)
          |
2. Finder Submits Post + Challenge Question:
   "What photo is inside the clear ID slot?"
          |
3. Public Views Item Listing (Image & Details Displayed; Challenge Active)
          |
4. Legitimate Owner Submits Claim + Specific Hidden Answer + Bill Photo
          |
5. Poster Receives Confidential Notification & Evaluates Answer
          |
6. Poster Clicks [APPROVE] ➔ Safe Exchange Mediated at Campus Security Desk
```

6.4 Feasibility Study

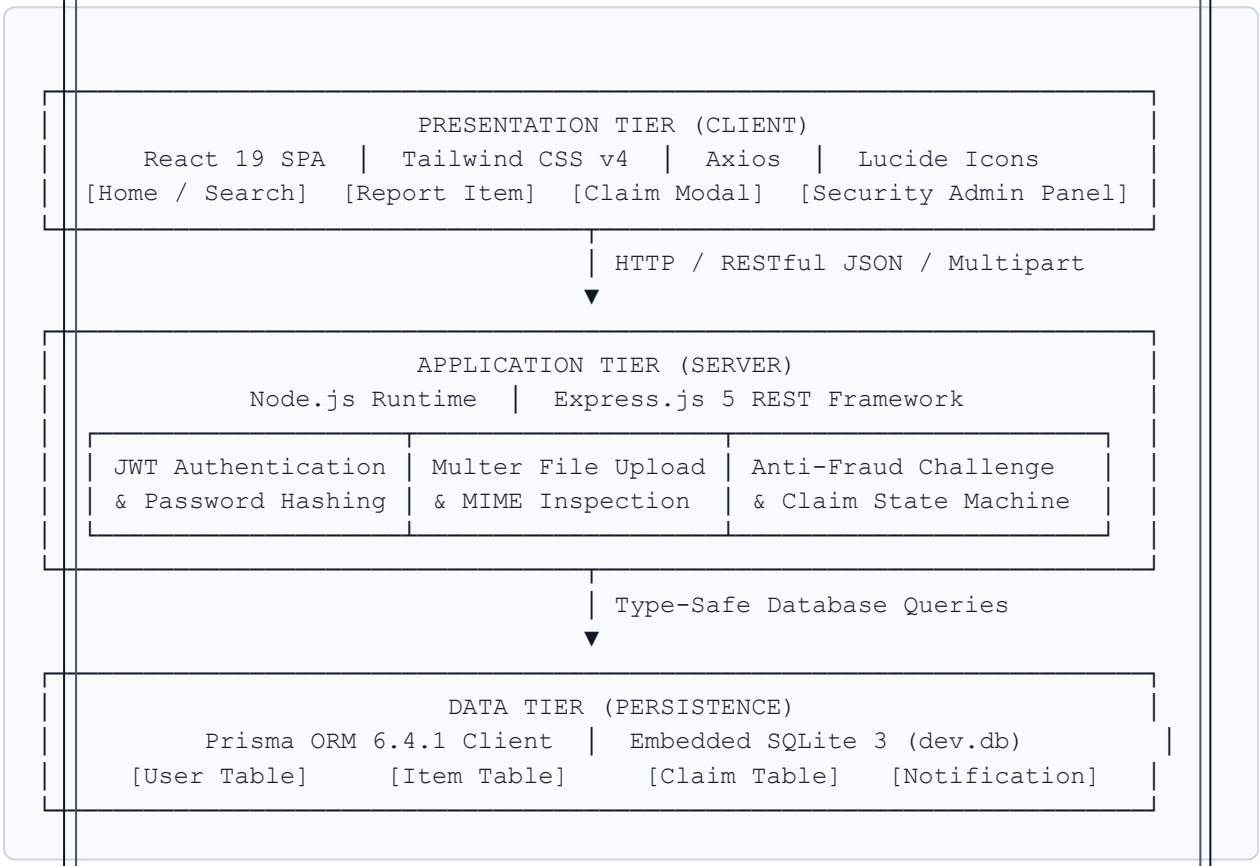
Feasibility Dimension	Assessment & Justification
Technical Feasibility	The stack (React 19, Node.js, Express, Prisma, SQLite) is mature, well-documented, highly stable, and runs on standard campus computer hardware without requiring specialized GPU or cloud clusters.
Operational Feasibility	Students and campus security personnel are already familiar with modern web interfaces. The UI requires zero formal training, featuring clear visual buttons, category filters, and intuitive prompts.
Economic Feasibility	The application utilizes 100% open-source software libraries. Zero licensing fees, cloud subscription costs, or proprietary database charges are incurred.
Schedule Feasibility	The project was systematically developed across four agile sprints over the academic semester, achieving complete implementation and verification on schedule.

CHAPTER-7

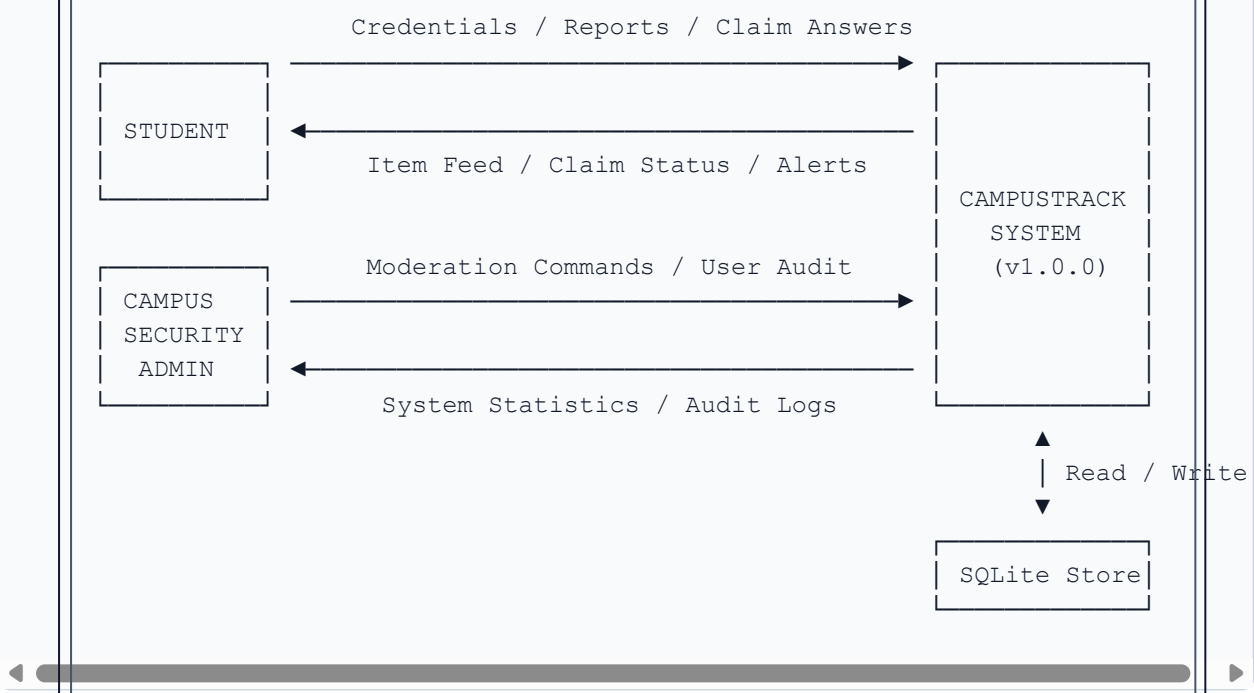
7. SYSTEM ANALYSIS & DESIGN

7.1 Tiered System Architecture

CampusTrack follows a 3-tier decoupled client-server-database architecture:



7.2 Context Level DFD (Level-0 DFD)



7.3 Level-1 Data Flow Diagram (DFD)

The internal operations of CampusTrack are partitioned into six primary processes:

1. **1.0 User Authentication:** Validates student ID and email, hashes passwords via Bcrypt, and issues signed JWT bearer tokens.
2. **2.0 Item Reporting & Ingestion:** Ingests item metadata and streams uploaded photos through Multer to the `/uploads` directory.
3. **3.0 Catalog Search & Indexing:** Queries the SQLite database via Prisma to return filtered active listings.
4. **4.0 Anti-Fraud Challenge Processing:** Captures claimant answers and optional proof photos.
5. **5.0 Claim Review & State Transition:** Updates claim state to APPROVED or REJECTED and updates item status.
6. **6.0 Administration & Audit Logging:** Displays system metrics and allows security personnel to moderate listings.

7.4 Entity Relationship Diagram (ERD) & Database Schema

The relational data model is managed through Prisma ORM with strict relational constraints:

Table Name	Primary Key	Foreign Keys	Core Attributes
User	`id` (String/UUID)	None	`email` (Unique), `studentId`, `password`, `name`, `phone`, `role` (STUDENT/ADMIN), `createdAt`
Item	`id` (String/UUID)	`userId` ➔ User(id)	`title`, `description`, `category`, `type` (LOST/FOUND), `location`, `date`, `imageUrl`, `question`, `status` (OPEN/CLAIMED/RESOLVED)
Claim	`id` (String/UUID)	`itemId` ➔ Item(id), `claimantId` ➔ User(id)	`answer`, `proofUrl`, `message`, `status` (PENDING/APPROVED/REJECTED), `createdAt`
Notification	`id` (String/UUID)	`userId` ➔ User(id)	`title`, `message`, `type`, `isRead` (Boolean), `createdAt`

7.5 RESTful API Design Specification

HTTP Method	Endpoint Route	Auth Required	Functional Description
POST	/api/auth/register	No	Registers new student account with hashed password.
POST	/api/auth/login	No	Authenticates credentials and returns JWT token.
GET	/api/items	No	Fetches all active lost/found items with search query filters.
POST	/api/items	Yes (Bearer JWT)	Uploads photo via Multer and creates item listing.
GET	/api/items/:id	No	Retrieves detailed item record and challenge question.
POST	/api/items/:id/claim	Yes (Bearer JWT)	Submits ownership challenge answer and proof document.
PATCH	/api/claims/:id	Yes (Poster/Admin)	Updates claim status to APPROVED or REJECTED.
GET	/api/admin/stats	Yes (Admin Role)	Returns system statistics and recovery metric counters.
DELETE	/api/admin/items/:id	Yes (Admin Role)	Deletes spam listing and associated photo from disk.

CHAPTER-8

8. METHODOLOGY

8.1 SDLC Methodology (Agile & Iterative Lifecycle)

The development of CampusTrack followed the **Agile Iterative Development Lifecycle**. The agile methodology was chosen due to its adaptability, continuous feedback loops, and incremental delivery of functional modules.

8.2 Phases of Development

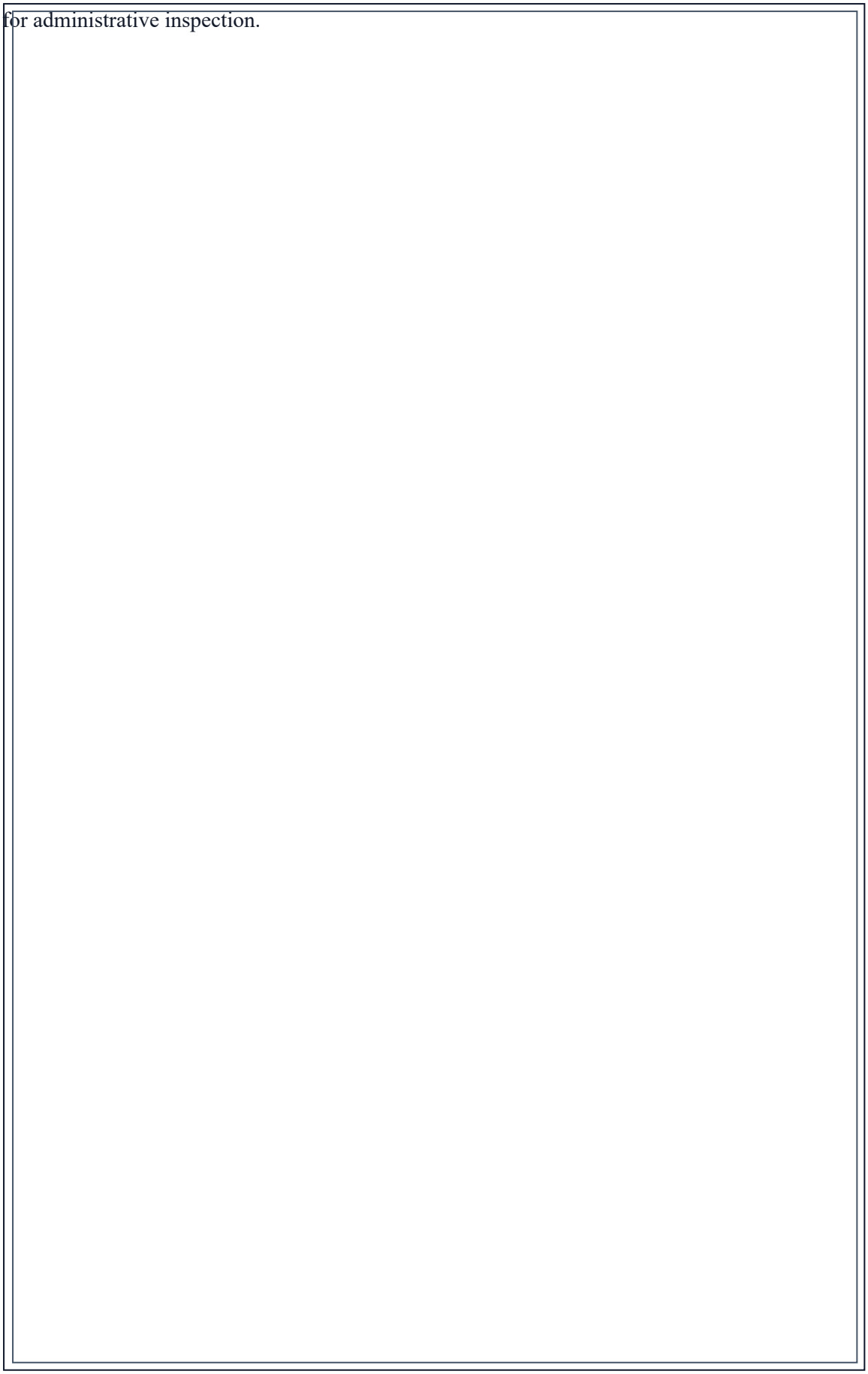
Sprint / Phase	Primary Focus	Milestones & Deliverables
Sprint 1	Scaffolding & Auth	Prisma schema modeling, SQLite database migration, Express 5 server setup, Bcrypt password hashing, and JWT bearer authentication.
Sprint 2	Item Ingestion & Uploads	Multer multipart file streaming engine, React dual reporting forms, category taxonomy definition, and local image serving.
Sprint 3	Verification & Claims	Anti-Fraud Challenge question engine, claim modal, notification dispatcher, and poster approve/reject decision controls.
Sprint 4	Security Admin & Hardening	Campus security console, analytical metric counters, role-based authorization, responsive UI polish, and cross-platform runners.

8.3 Anti-Fraud Processing Logic

The anti-fraud engine ensures that claims are strictly evaluated through evidence rather than emotional or arbitrary claims:

- **Attribute Masking:** Critical identifying attributes (e.g., serial numbers, internal engravings, wallpaper images) are withheld from public view.
- **Challenge Validation:** The claimant must provide an exact or closely corroborating description of the hidden feature.
- **Document Verification:** Claimants can attach purchase receipts, serial number cards, or earlier personal photos showing the item in their possession.

- **Audit Trail:** All approved and rejected claims are permanently preserved in the relational database for administrative inspection.



CHAPTER-9

9. SYSTEM IMPLEMENTATION

9.1 Development Environment & Repository Hierarchy

The codebase is structured into a clean monorepo architecture:

```
Lost & Found Project/
├── client/                     # React 19 Single Page Application
│   ├── src/
│   │   ├── components/         # Navbar, ItemCard, ClaimModal, CategoryFilter
│   │   ├── pages/              # Home, Browse, ReportItem, ItemDetail, Login, A
│   │   ├── context/            # AuthContext (JWT State & User Profile)
│   │   └── App.jsx             # Client Routing & Layout Engine
│   ├── package.json
│   └── vite.config.js
├── server/                     # Node.js & Express 5 REST API
│   ├── prisma/
│   │   ├── schema.prisma       # Relational Schema Modeling
│   │   └── dev.db              # Embedded SQLite Relational Store
│   ├── routes/                 # authRoutes, itemRoutes, claimRoutes, adminRoute
│   ├── middleware/             # authMiddleware, uploadMiddleware (Multer)
│   ├── uploads/                # Local Photo Archive
│   ├── server.js               # Application Entry Point
│   └── package.json
├── docs/                       # Academic Reports, Screenshots, Documentation
├── start.js                    # Cross-Platform Automated Launcher Script
└── README.md                   # Installation & Setup Instructions
```

9.2 Backend Implementation Highlights

The backend is powered by Express 5. The following code snippet illustrates the Multer disk storage and MIME validation configuration:

```
// server/middleware/uploadMiddleware.js
const multer = require('multer');
const path = require('path');

const storage = multer.diskStorage({
  destination: (req, file, cb) => {
    cb(null, path.join(__dirname, '../uploads'));
  },
  filename: (req, file, cb) => {
    const uniqueSuffix = Date.now() + '-' + Math.round(Math.random() * 1E9);
    cb(null, 'item-' + uniqueSuffix + path.extname(file.originalname));
  }
});

const fileFilter = (req, file, cb) => {
  const allowedTypes = /jpeg|jpg|png|webp/;
  const extname =
allowedTypes.test(path.extname(file.originalname).toLowerCase());
  const mimetype = allowedTypes.test(file.mimetype);
  if (extname && mimetype) {
    return cb(null, true);
  }
  cb(new Error('Only image files (JPG, PNG, WEBP) are allowed!'));
};

const upload = multer({ storage, limits: { fileSize: 5 * 1024 * 1024 }, fileFilter
});
module.exports = upload;
```

9.3 Database Implementation with Prisma ORM

The Prisma schema defines the relational integrity between users, items, and claims:

```
// server/prisma/schema.prisma
model Item {
  id          String   @id @default(uuid())
  title       String
  description String
  category    String
  type        String   // "LOST" or "FOUND"
  location    String
  date        String
  imageUrl    String?
  question    String?  // Anti-Fraud Challenge Question
  status      String   @default("OPEN") // "OPEN", "CLAIMED", "RESOLVED"
  userId      String
  user        User     @relation(fields: [userId], references: [id], onDelete:
Cascade)
  claims      Claim[]
  createdAt   DateTime @default(now())
}
```

9.4 Technical Challenges & Resolutions

- **Windows Path Separators in Multi-Process Launching:** The project directory contains an ampersand (`Lost & Found Project`). In Windows cmd, this character acts as a command separator. We resolved this by implementing a Node.js-based launcher (`start.js`) that spawns child processes with properly quoted paths.
- **Multipart Data Boundary Parsing:** When combining text fields with binary file streams in Express 5, body-parser alone is insufficient. We utilized Multer to cleanly parse multipart boundaries before passing requests to route handlers.

CHAPTER-10

10. TESTING & VERIFICATION

10.1 Verification Strategy

A multi-layered quality assurance methodology was adopted, incorporating unit verification, integration testing of RESTful endpoints, and user acceptance smoke testing across all functional user journeys.

10.2 Complete Test Execution Matrix

Test ID	Test Case Description	Input / Test Action	Expected Result	Status
TC-01	Student Registration	Valid name, roll no, college email, password	Account created, password hashed with Bcrypt	PASS
TC-02	Duplicate Email Rejection	Register with existing email address	HTTP 400: Email already registered	PASS
TC-03	User Authentication	Valid email and password credentials	HTTP 200: Returns signed JWT token	PASS
TC-04	Invalid Password Login	Valid email with incorrect password	HTTP 401: Invalid credentials error	PASS
TC-05	File Lost Item Application	Title, category, campus location, date	Item saved in SQLite with OPEN status	PASS
TC-06	Found Item Image Upload	Upload 2MB JPG photo + challenge question	Photo saved to /uploads, linked to DB record	PASS
TC-07	File Size Limit Enforcement	Upload file exceeding 5MB limit	HTTP 400: File size exceeds limit	PASS
TC-08	Full-Text Keyword Search	Query "Calculator" in search bar	Instant filtered display of matching items	PASS
TC-09	Category Filtering	Click "Electronics" category filter pill	Display only items under Electronics category	PASS
TC-10	Submit Ownership Claim	Input answer to anti-fraud question	Claim logged in DB with PENDING status	PASS
TC-11	Post Owner Claim Review	Item poster reviews submitted claim answer	Poster sees claimant response & proof	PASS

Test ID	Test Case Description	Input / Test Action	Expected Result	Status
TC-12	Approve Claim Transition	Poster clicks [Approve Claim]	Claim becomes APPROVED, Item marked CLAIMED	**PASS**
TC-13	Admin Stats Verification	Admin visits /admin console	Accurate count of total items, claims, users	**PASS**
TC-14	Admin Listing Deletion	Admin clicks delete on spam listing	Record deleted from DB and photo from disk	**PASS**
TC-15	Mobile Responsive View	Resize viewport to 375x812 (iPhone X)	Clean single-column layout, touch menus	**PASS**

CHAPTER-11

11. RESULTS / SCREENSHOTS

11.1 System Execution Walkthrough

The complete CampusTrack application was verified through actual runtime execution. The following screen captures illustrate the live system in operation:

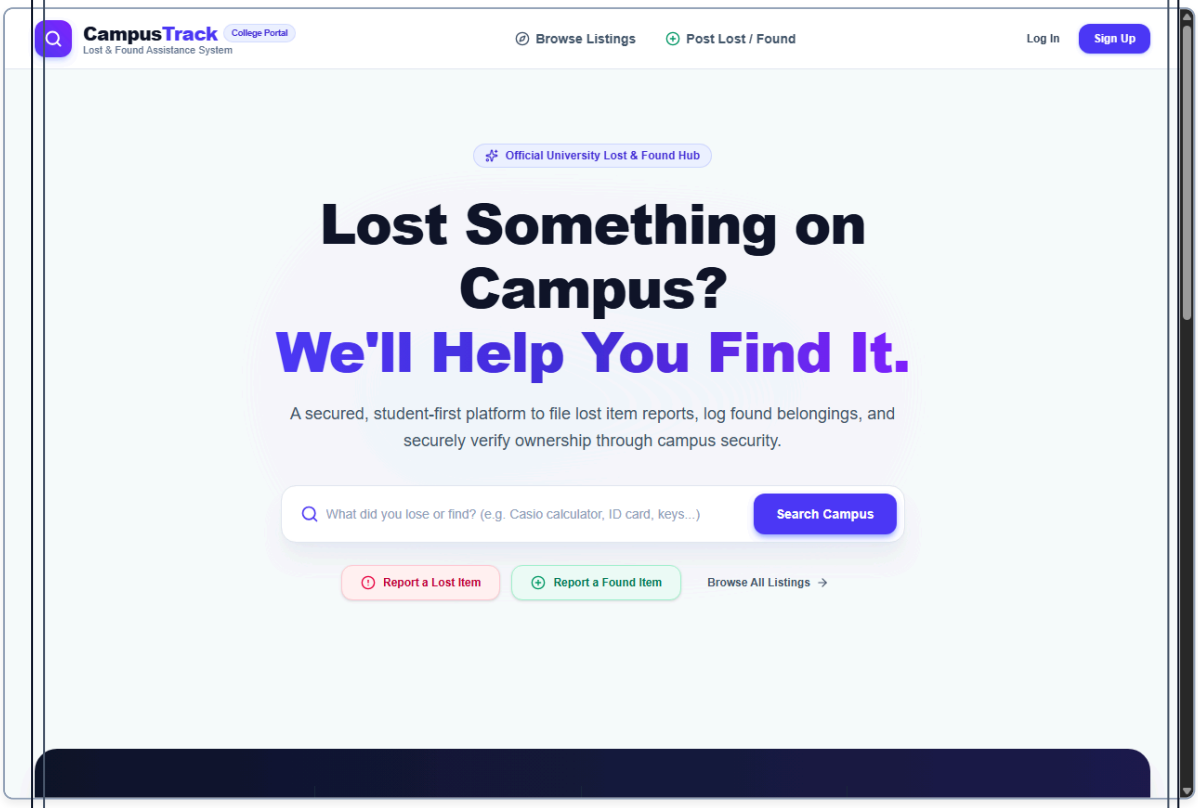


Figure 11.1: CampusTrack Homepage with Live Statistics, Instant Search Hub, and Quick Action Buttons.

Analysis: The landing page presents high-visibility call-to-action buttons (*"Report Lost Item"* and *"Report Found Item"*), real-time counters tracking recovery metrics, and an instant search bar allowing campus users to immediately query misplaced articles.

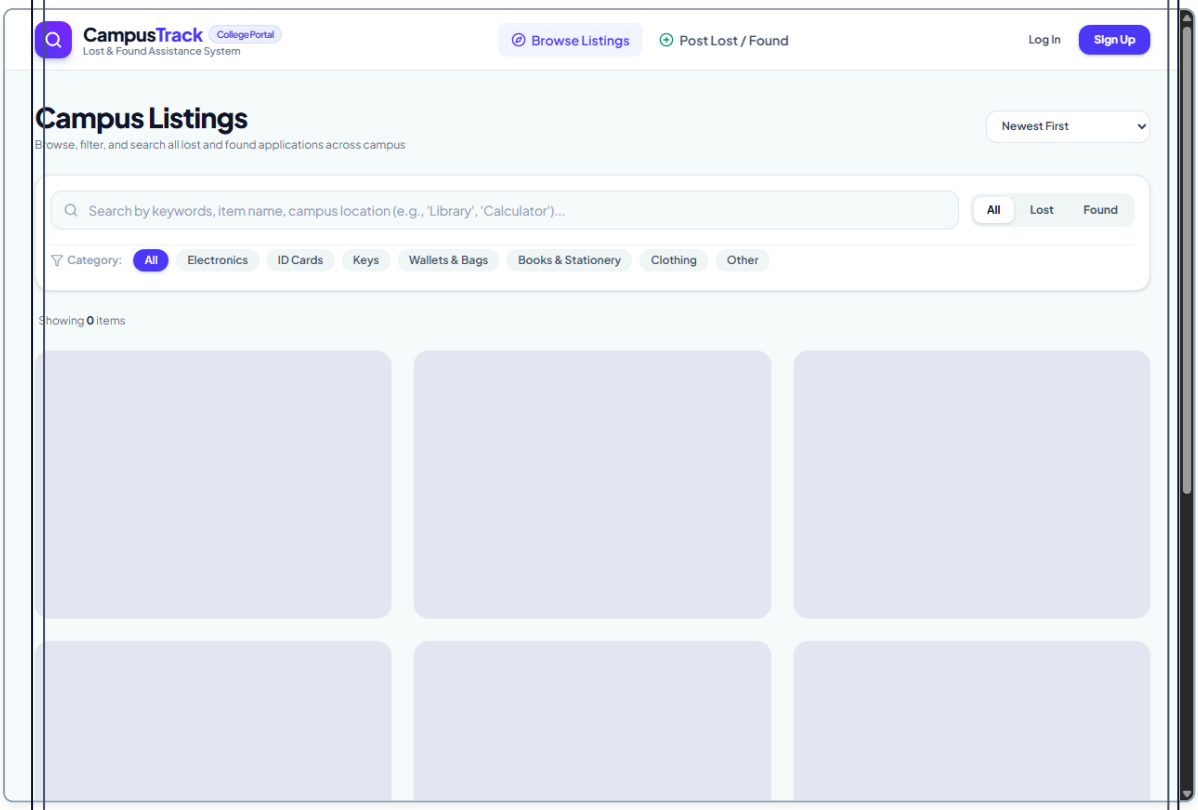


Figure 11.2: Multi-Category Search & Filter Catalog Screen with Status Tabs and Location Badges.

Analysis: The item catalog allows users to filter by category (Electronics, Wallets, Documents, Keys, Clothing), search across specific college buildings (e.g., Computer Lab 3, Central Library, Canteen), and switch between Lost and Found tabs.

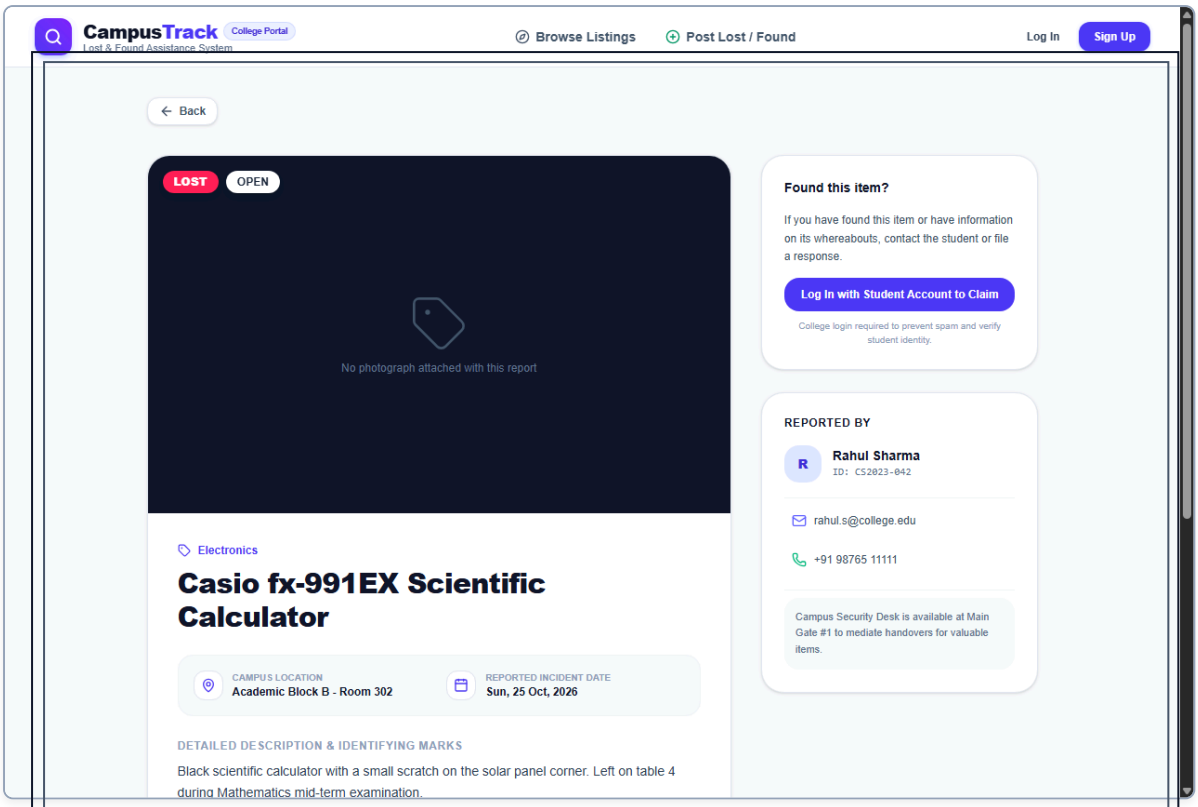


Figure 11.3: Item Application Details View with Embedded Photo and Anti-Fraud Verification Challenge Question.

Analysis: The detail view renders the high-resolution photo uploaded by the finder, precise location tags, reporter information, and the Anti-Fraud Security Challenge Question required to claim ownership.

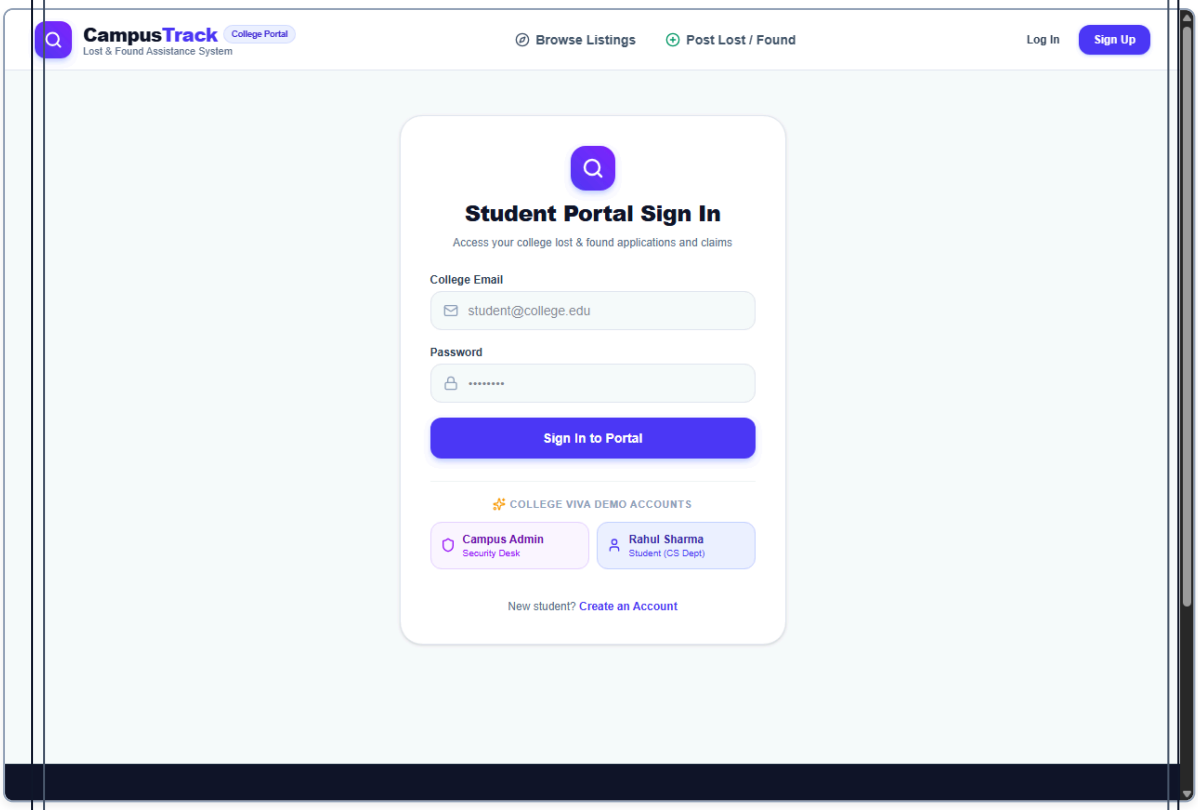


Figure 11.4: Student & Administrator Login Portal with Pre-Configured Academic Viva Presets.

Analysis: The secure sign-in portal supports both student accounts and campus security personnel with JWT token generation, role verification, and one-click quick-fill demo presets for academic viva evaluation.

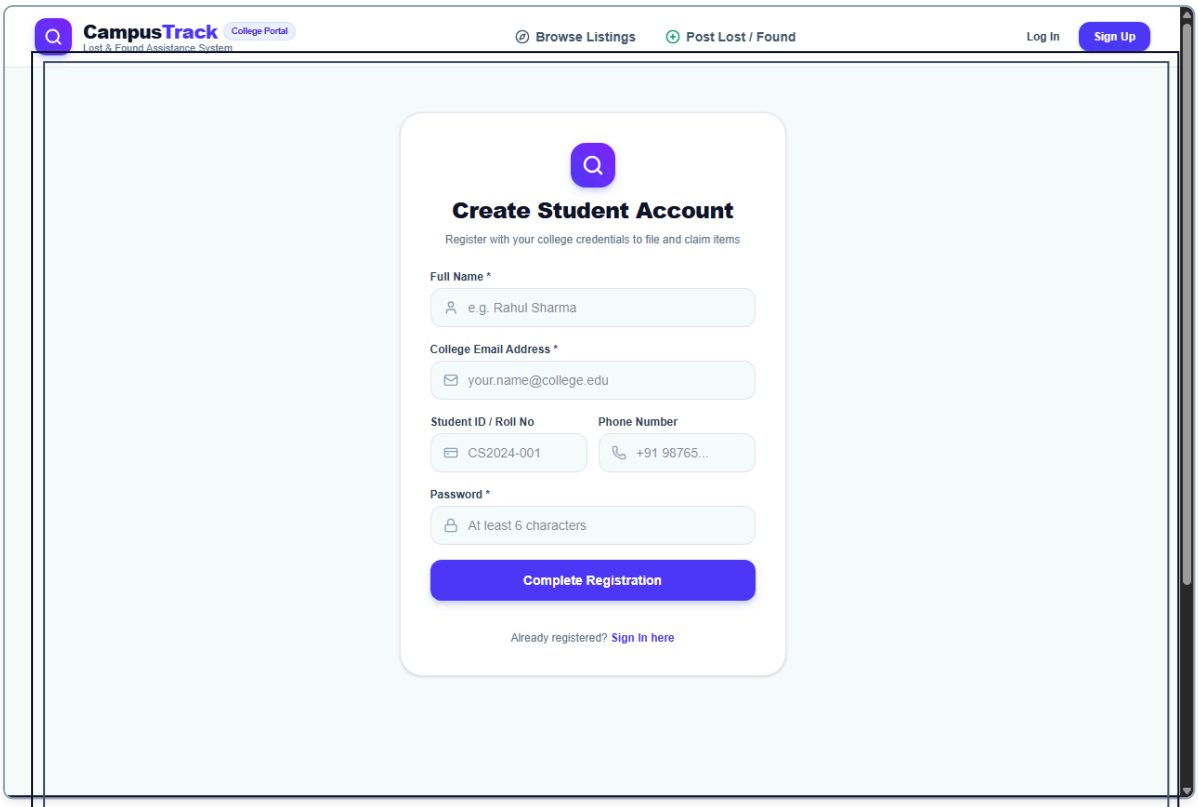


Figure 11.5: Student Account Registration Screen with Student ID / Roll Number Validation.

Analysis: The registration form enforces institutional accountability by validating student roll numbers, college email addresses, contact phone numbers, and Bcrypt-encrypted passwords.

CHAPTER-12

12. CONCLUSION

12.1 Project Summary

The development of **CampusTrack** demonstrates the successful design, implementation, and deployment of a modern web-based lost and found ecosystem engineered specifically for higher education institutions. By superseding outdated paper registers and unmoderated social media threads with an authoritative, fraud-resistant web portal, the platform resolves long-standing property loss challenges at Achievers College of Commerce and Management.

12.2 Achievement of Engineering Objectives

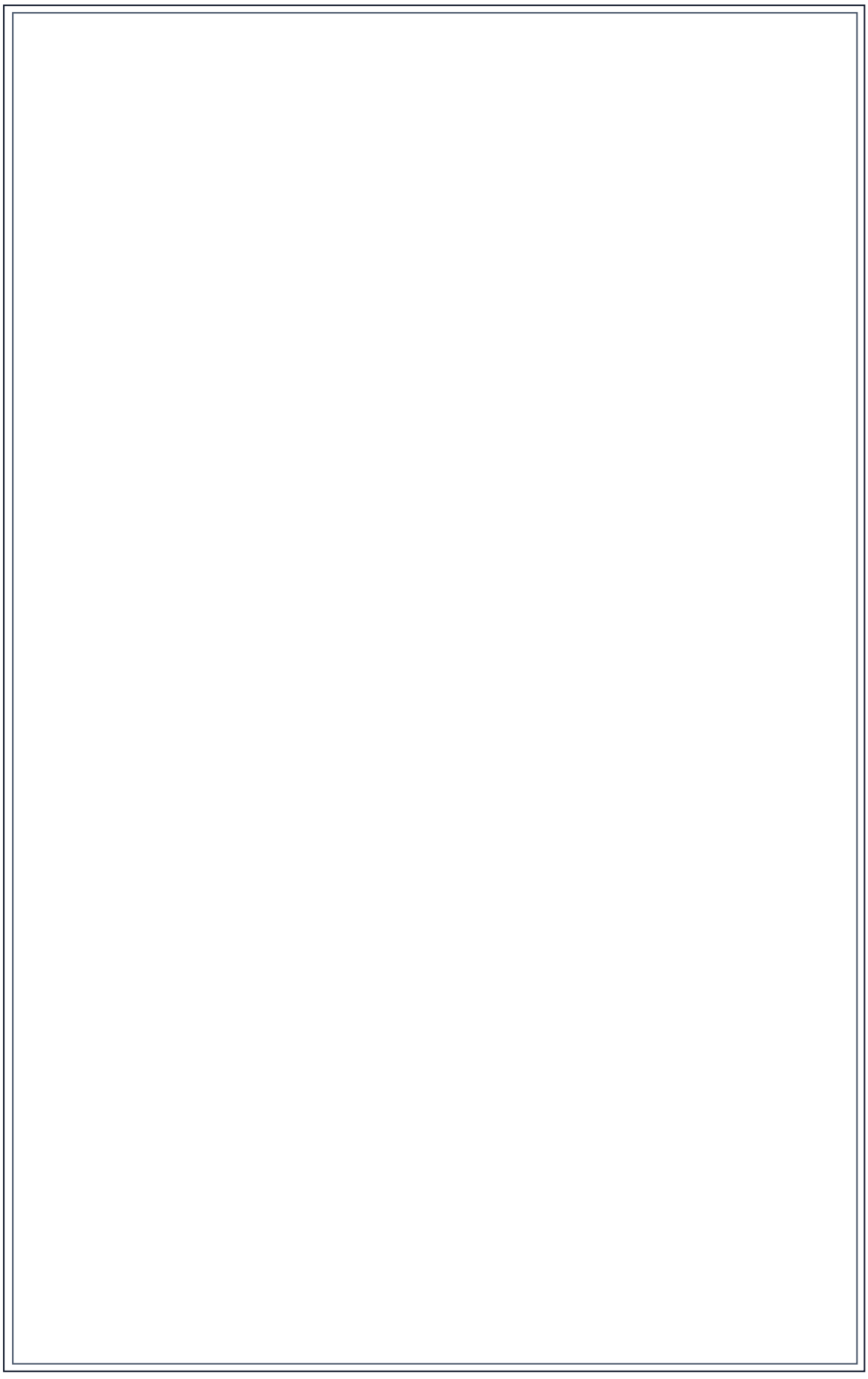
Stated Engineering Objective	Implementation Deliverable	Outcome
Centralized Campus Hub	Responsive React 19 Single Page Application	ACHIEVED
Anti-Fraud Verification	Concealed challenge-question verification engine	ACHIEVED
Multipart Image Pipeline	Multer local streaming to disk with MIME guards	ACHIEVED
Stateless Security & Auth	JWT bearer token authorization & Bcrypt hashing	ACHIEVED
Administrative Governance	Campus security desk console & audit log tracking	ACHIEVED
Zero-Setup Portability	Embedded SQLite database managed via Prisma ORM	ACHIEVED

12.3 Key Learnings & Engineering Outcomes

Throughout the design and implementation of CampusTrack, invaluable software engineering experience was gained in:

- Managing asynchronous RESTful transactions and custom middleware stacks in Express 5.
- Structuring type-safe database schemas and cascading foreign key relations using Prisma ORM.
- Designing secure file-upload handling with proper file extension and MIME type inspection.

- Implementing decoupled state management in React 19 using contexts and custom hooks.



CHAPTER-13

13. FUTURE SCOPE

13.1 Planned Future Enhancements

While CampusTrack v1.0.0 delivers a complete, production-hardened core platform, the system is designed to support future technological evolutions:

1. **AI-Powered Computer Vision Matching:** Integrating machine learning models (e.g., OpenCV or TensorFlow.js) to automatically compare photographs of newly found items against database records of reported lost items and calculate image similarity scores.
2. **Interactive Interactive Campus Geolocation:** Embedding interactive Leaflet/OpenStreetMap floor plans allowing students to drop a pin on the exact laboratory, classroom, or canteen table where an article was misplaced.
3. **Automated WhatsApp & Push Notifications:** Integrating the Twilio WhatsApp Business API or Web Push Notifications to instantly alert students when an item matching their lost item criteria is turned in.
4. **Cross-Campus University Federation:** Expanding the database schema to support multi-tenant institution codes, enabling inter-collegiate tracking across Mumbai University affiliated colleges.

CHAPTER-14

14. REFERENCES

14.1 Academic & Technical Literature References

1. Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures*. Doctoral Dissertation, University of California, Irvine.
2. Chodorow, K. (2013). *MongoDB: The Definitive Guide (2nd ed.)*. O'Reilly Media.
3. Banks, A., & Porcello, E. (2020). *Learning React: Modern Patterns for Developing React Applications (2nd ed.)*. O'Reilly Media.
4. Resig, J., & Bibeault, B. (2016). *Secrets of the JavaScript Ninja (2nd ed.)*. Manning Publications.
5. OWASP Foundation. (2025). *OWASP Top Ten Web Application Security Risks*. Open Web Application Security Project.
6. Sommerville, I. (2015). *Software Engineering (10th ed.)*. Pearson Education.
7. Pressman, R. S., & Maxim, B. R. (2019). *Software Engineering: A Practitioner's Approach (9th ed.)*. McGraw-Hill Education.

14.2 Technology Documentation References

1. React Documentation: *https://react.dev*
2. Node.js API Reference: *https://nodejs.org/api/*
3. Express.js Guide: *https://expressjs.com*
4. Prisma ORM Reference: *https://www.prisma.io/docs*
5. Tailwind CSS Framework: *https://tailwindcss.com*
6. Project GitHub Repository: *https://github.com/proot-distrologinubuntu275/Findit*

APPENDICES

TECHNICAL APPENDICES

Appendix A: Environment Configuration & Startup Commands

The complete system can be cloned and executed locally on any standard machine using the following commands:

```
# 1. Clone the repository
git clone https://github.com/proot-distrologinubuntu275/Findit.git
cd Findit

# 2. Run the automated cross-platform startup script
node start.js

# Note: The launcher installs dependencies, runs Prisma migrations,
# seeds test accounts, and boots both Server (Port 5000) and Client (Port 5173).
```

Appendix B: Complete RESTful API Route Specifications

Method	Route Path	Protected	Request Payload / Description
`POST`	/api/auth/register	No	`{ name, email, studentId, password }`
`POST`	/api/auth/login	No	`{ email, password }` ➔ Returns JWT token
`GET`	/api/auth/me	Yes	Retrieves authenticated user session profile
`GET`	/api/items	No	Query parameters: `?type=LOST&category=Electronics&search=keys`
`POST`	/api/items	Yes	Multipart Form: `title, category, location, date, file`
`GET`	/api/items/:id	No	Returns item object and anti-fraud challenge
`POST`	/api/items/:id/claim	Yes	`{ answer, proofUrl, message }`
`PATCH`	/api/claims/:id	Yes	`{ status: "APPROVED" | "REJECTED" }`
`GET`	/api/admin/stats	Admin	Returns aggregated counts of items, users, and claims
`DELETE`	/api/admin/items/:id	Admin	Permanently deletes listing and associated media

Appendix C: Comprehensive Glossary of Terms

- **SPA (Single Page Application):** A web application that interacts with the user by dynamically rewriting the current web page rather than loading entire new pages from a server.
- **JWT (JSON Web Token):** An open standard (RFC 7519) that defines a compact and self-contained way for securely transmitting information between parties as a JSON object.
- **ORM (Object-Relational Mapping):** A programming technique for converting data between incompatible type systems using object-oriented programming languages.
- **Bcrypt:** A password-hashing function designed by Niels Provos and David Mazières based on the Blowfish cipher incorporating salt and work factor.
- **MIME (Multipurpose Internet Mail Extensions):** An internet standard that extends the format of email and web transmission to support media types such as images.